

B & B+ AĞAÇLARI

Modern Veri Sistemlerinin Görünmez Mimarları

Hazırlayan: İsa Çiçek
Veri Organizasyonu Ödevi

Temel Yapı ve Farklar

B ve B+ Ağaçlarını Birbirinden Ayıran Mimari
Özellikler

B vs B+ Ağacı: Mimari Fark

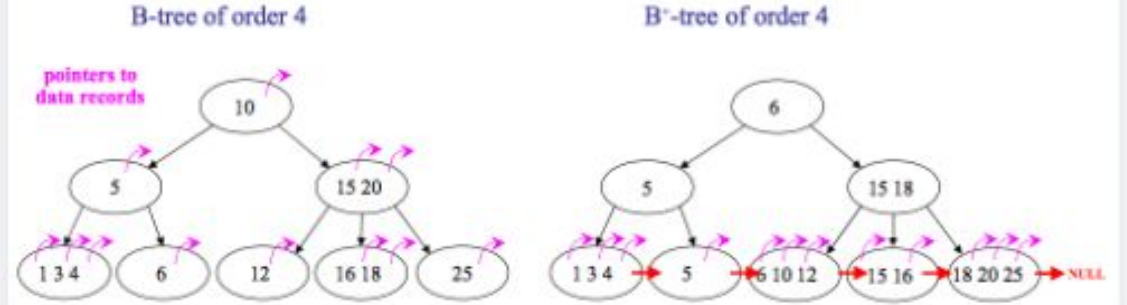
Yapısal Ayrım

B-Ağacı: Veriler hem iç düğümlerde hem de yapraklarda saklanır. Aranan veriye üst seviyelerde ulaşırsa arama erken biter.

B+ Ağacı: Veriler **sadece yapraklarda** saklanır. İç düğümler sadece "navigasyon" amaçlı yönlendirici anahtarları barındırır.

En kritik fark; B+ ağacında tüm yaprakların birbirine **bağlı liste** (linked list) ile bağlı olmasıdır. Bu, aralık sorgularını mükemmelleştirir.

- A B⁺-tree can be viewed as a B-tree in which each node contains only keys (not pairs), and to which an additional level is added at the bottom with linked leaves



Somut Analoji: Hastane Arşivi



Hiyerarşik Düzen

B-Tree: Her katta hem yönlendirici tabela hem de hasta dosyası vardır. Kat kat gezmeniz gerekir.

B+ Tree: Danışmadaki rehber sizi sadece doğru odaya ve rafa yönlendirir (İç Düğüm). Tüm dosyalar sadece en alt kattaki raftadır (Yaprak). Raftaki tüm dosyalar birbirine iplerle bağlıdır; birini bulduğunuzda sonrakine bakmak için danışmaya dönmenize gerek kalmaz.

Karşılaştırma Tablosu

Özellik	B-Ağacı (B-Tree)	B+ Ağacı (B+ Tree)
Veri Saklama	Tüm düğümlerde (İç + Yaprak)	Sadece yaprak düğümlerde
Arama Süreci	Değişken (Erken bitebilir)	Sabit (Her zaman yaprağa iner)
Sıralı Erişim	Zor (Sürekli ağaç dolaşma)	Çok Hızlı (Yapraklar arası link)
Silme İşlemi	İç düğümlerde karmaşıktır	Sadece yaprakta olduğu için basittir

Neden Modern Sistemlerde Tercih Edilir?



Disk Dostu Mimari

CPU ve RAM çok hızlıdır, ancak diske erişmek (I/O) her zaman darboğazdır.

B+ ağaçlarında iç düğümlerde veri saklanmadığı için tek bir disk bloğuna (sayfa) daha fazla anahtar sığdırılır.

Bu "fan-out" (dallanma) oranını artırır, ağacın boyunu kısaltır ve milyarlarca kayıt içinde sadece 3-4 disk okuması ile sonuca ulaşılmasını sağlar.

Veri Tabanı Devlerinde Kullanım



MySQL (InnoDB)

Birincil anahtarlar için **B+ Tree** kullanır. Veriler yapraklarda fiziksel olarak sıralıdır (Clustered Index). Aralık sorgularını saniyeden kısa sürede çözer.



PostgreSQL

Standart indeks tipi olarak B-Tree kullanır, ancak bu mimari kaputun altında sıralı erişim için modifiye edilmiş yüksek performanslı bir B+ Tree yapısıdır.



MongoDB

Modern sürümlerinde WiredTiger motoru ile veri indeksleme için **B+ Tree** yapılarını kullanır. Rastgele okuma ve yazma işlemlerini dengelemek için idealdir.

Dosya Sistemlerinde Rolü

İşletim sistemleri, disk üzerindeki milyonlarca klasör ve dosyayı yönetmek için bu yapılara muhtaçtır.

NTFS (Windows): Klasör yapıları ve dosya niteliklerini hızlıca aramak için B-Tree türevlerini kullanır.

XFS & EXT4 (Linux): Büyük dizinleri ölçeklendirmek ve disk parçalanmasını önlemek için B+ Tree ve Htree varyantlarına güvenir.



Büyük Veri ve Arama Motorları

Linear Scan (Milyon Veri)

1.000.000 Disk Okuma

Binary Tree

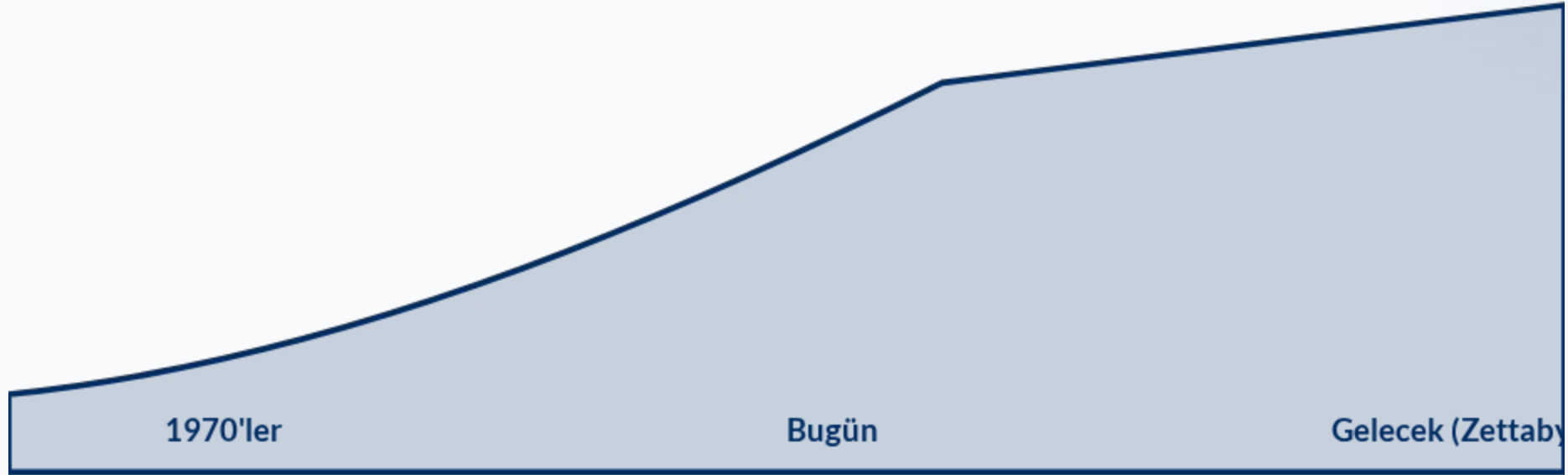
~20 Disk Okuma

B+ Tree (Mertebe: 100)

3 Disk
Okuma

Arama motorları, diske erişim maliyetini minimize etmek ve logaritmik zaman karmaşıklığını korumak için bu hiyerarşik yapıları kullanmak zorundadır.

Kişisel Yorum: Gelecekte Kullanılacak mı?



Evet, kesinlikle kullanılacak. SSD ve NVMe hızları artsa da üretilen veri hacmi çok daha hızlı büyüyor.

Veriyi bloklar halinde hiyerarşik olarak indeksleme mantığı (özellikle veri okuma performansı söz konusu olduğunda) IT dünyasının temel taşı olmaya devam edecektir.

Dinlediğiniz İçin Teşekkürler

Sorularınız varsa yanıtlamaktan memnuniyet
duyarım.

İsa Çiçek

Image Sources



Thumbnail for
stackoverflow.com

<https://i.sstatic.net/l6UyF.png>

Source: stackoverflow.com



Thumbnail for
storageaspects.co.uk

<https://storageaspects.co.uk/wp-content/uploads/2023/01/SASU100919-Medical-Records-Storage-2.jpg>

Source: storageaspects.co.uk



Thumbnail
for
canovate.com

<https://canovate.com/wp-content/uploads/2024/08/Sunucu-Odasi-Tasarimi-Modern-Cozumler-ve-En-Iyi-Uygulamalar-1170x646.webp>

Source: canovate.com